

Kiroween 2026 · solo entry by Fernando Lizana · built with Kiro, spec by spec

Live: https://necro-oracle.vercel.app · **Code:** https://github.com/FernandoLizana/necro-oracle (MIT, `.kiro/` committed) · **Demo video:** https://youtu.be/LqITM5tS1Oo

Category: Frankenstein (it also lands squarely on the Resurrection and Costume Contest themes, explained below)

☠ Judges' quick path (60 seconds)

1. Open **https://necro-oracle.vercel.app**. A 1986 boot sequence plays, and any key skips it.
2. On the Game Boy-style menu, press **Enter** (or **A**) on `tomaes/pytarl`. The oracle summons the repo and lays a **Fates** spread of Major Arcana. Each card comes from a real tree-sitter diagnosis with evidence.
3. Type `explain 2` (or `explain present` — `past`, `present` and `future` are aliases of positions 1, 2 and 3) to open the evidence scroll behind the Present card (file:line, the actual finding).
4. Type `resurrect` (or click **RESURRECT**). The Necronomicon ritual plays, then the Python 2 to 3 diff, the vitality delta and a downloadable patch appear. On a live `tomaes/pytarl` run (2026-11-09) the deterministic Pre_Pass alone raised vitality **0 → 12**, cleared Judgement and finished in ~3 s with zero LLM residuals and **no LLM cost**.
5. Type `lovers pytarl.py js` (or click **LOVERS** — Judgement now links the `lovers-js` grimoire, so the chip appears after a successful resurrect). The Frankenstein binding returns a Python heart, a JS beloved, a JSON contract and tests on both sides. A later live run of `lovers pytarl.py javascript` bound **7 functions** for **6,418 tokens** (**0.0842 USD**); an earlier run on the same file routed to **STRONG** (`anthropic/claude-sonnet-4.5`) for **0.019 USD**.
6. Type `tarot will my legacy code live?`. Three of the 22 Major Arcana are dealt with a ceremonial reveal and a **deterministic offline reading** (the model-backed `/api/tarot` text is on the Roadmap, not live yet).
7. Type `demo lich-py2`. Fully offline recorded replay: reading, resurrect and Lovers all work with the Mock LLM, no network or API budget required.
8. Type `key status`. Bring-your-own OpenRouter key mode shows only the last four characters, and the key lives only in this tab's memory.

Prefer clicking? Every command has a button in the bottom row. Add `?ritual=force` to the URL to always see the full ritual animation.

Inspiration

Everyone has a graveyard repository: a Python 2 tool someone wrote in 2012 that "still works, don't touch it," a side project whose CI died with Travis, a library nobody dares upgrade. Migrating that code is real, unglamorous, risky work. Static analyzers produce walls of warnings nobody reads, and "just ask an LLM to rewrite it" produces confident code nobody can trust.

I wanted a tool that makes legacy-code triage **memorable and trustworthy at the same time**. Tarot gave me the right metaphor. A reading is a small, ordered set of cards, each with a position and a meaning, and orientation (upright or reversed) carries information. That turns out to be a great information architecture for a code-health report: a few cards that matter, laid out as a story (Past, Present, Future), instead of 400 lint lines.

Then I added the Halloween twist. The oracle doesn't stop at telling you your code is dead. **She raises it.**

What it does

The Necro-Oracle is a retro CRT terminal styled as a Game Boy (DMG) cartridge from a haunted 1986: a strict 4-shade phosphor palette (`#001a00 #0b4f1a #2fa84a #8cff9e`), pixel-art Major Arcana, scanlines and an original chiptune theme. Behind the costume is a serious engineering pipeline:

1. **Summon:** `summon owner/repo` pins any public GitHub repository to an exact commit SHA and downloads it as untrusted data (it is parsed, never executed).
2. **Read:** `read fates | necropsy | celtic` runs a **12-card diagnosis deck**, one deterministic detector per card (tree-sitter AST, manifests, OSV advisories):

Card	Diagnosis
XX Judgement	Dead runtime: Python 2 syntax and Py2-only manifests
XVI The Tower	Rotting or broken dependencies
XV The Devil	Security sins: masked secret patterns, dangerous calls, known OSV vulnerabilities
XIII Death	Dead code
X Wheel of Fortune	Missing or rusted tests
IX The Hermit	Missing or stale docs
XII The Hanged Man	Abandonment
XIV Temperance	No CI, linting or formatting
XI Justice	Licence or governance gaps
XVIII The Moon	Tangled complexity
XVII The Star	Remaining strengths (always drawn, so a reading is never purely bleak)
XXI The World	Overall health

Upright = active problem, reversed = latent risk. Each card shows its evidence (file, line, finding), and the reading ends with a 0–100 **vitality** gauge. The product rule, enforced in code, is *"the cards cannot lie"*: a deterministic detector decides whether a card is drawn, and the LLM never does. 3. **Explain:** `explain <position|card>` opens the evidence scroll behind any card. Positions may also be named `past`, `present` or `future` (aliases of 1, 2 and 3). 4. **Resurrect:** `resurrect [card|file]` revives Python 2 files to Python 3. A deterministic **Pre_Pass** of six idempotent fixers runs first (print statements, `except E, e:`, `<>`, long literals, dict iteration methods, unicode/str). The LLM only sees the **residual diagnostics** the fixers could not resolve, chunked so each prompt contains only the relevant part of the file. Every output must pass a validation gate (tree-sitter parse with zero error nodes, plus an AST check that no Python 2 construct survives) or it is rejected. You then see the Necronomicon ritual, a unified diff, the before/after vitality and a downloadable patch. 5. **The Lovers (Frankenstein binding):** `lovers <file> js` stitches a resurrected Python 3 file to a **JavaScript client**. Judgement draws the `lovers-js` grimoire, so after a successful resurrect the **LOVERS** chip is reachable in the button row. A deterministic AST template always emits the Python "heart" (a JSON-lines dispatcher). A routed LLM generates the JS "beloved" (a client that spawns the heart, never a reimplementation), a JSON contract schema and contract tests on both sides. All of it is checked with tree-sitter for both grammars plus a static deny-list scan (no `eval`, network, or fs writes) and delivered as a ZIP. 6. **Tarot (ceremonial draw):** `tarot [question]` draws three distinct cards from the **full 22-card Major Arcana** with a crypto RNG (Past, Present, Future) and gives a staged, ceremonial reveal plus a **deterministic offline reading**. The one-call FAST model-backed reading service is already built and tested in `@necro/coven`, but wiring it to a live `/api/tarot` route is deferred to the Roadmap — docs never claim that path is live. 7. **BYOK:** `key set | status | clear` and `model fast|strong <id>` let a visitor use their **own OpenRouter key** for this tab only. 8. **Delight and access:** a guided clickable menu, offline demo replays with autoplay (`demo lich-py2 | mixed-ruins | green-garden`) that cover reading, resurrect and Lovers under the Mock LLM, runtime-synthesized music (no audio files), and full keyboard, screen-reader and reduced-motion support.



The DMG landing menu: legacy repos, Seer, card backs



A Fates reading: evidence-backed cards and pixel arcana

How we built it with Kiro

The whole project was built in Kiro over **86 commits (Nov 3–9, 2026)**, with follow-up UX polish through `be1c9ae` and `5930ae6` (the commit this write-up is pinned to). The commit history reads as a progression from one spec to the next: `Spec 01 tasks 2.1, 3, 10 …` through `feat(judge-ready-pass): …` and the later layout fixes. All counts below come from the committed `.kiro/` directory.

Spec-driven development: 10 spec folders

#	Spec	Flow	Requirements	Acceptance criteria	Task items	Named correctness properties
1	`01-oracle-deck-engine`: cards, detectors, spreads, vitality	requirements-first	21	129	89	15
2	`02-github-summoner`: pinned-SHA intake, tarball caps, OSV/registry	requirements-first	10	101	128	33
3	`03-crt-terminal`: parser, reducer, NDJSON streaming, a11y, CSP	requirements-first	19	215	97	29
4	`resurrection-engine`: Pre_Pass, grimoire, coven runner, diff/patch	requirements-first	16	99	82	16
5	`visual-necromancy`: boot, ASCII cards, glitch, rot-and-rebirth	requirements-first	13	92	67	13
6	`arcana-visions`: pixel-art cards, Necronomicon ritual, Seer, music	requirements-first	13	86	53	16
7	`fix-wasm-tracing`: package tree-sitter WASM into Vercel functions	bugfix task	—	—	1	—
8	`oracle-landing-polish`: DMG game-menu landing	**design-first**	32	111	65	12
9	`lovers-binding`: OpenRouter layer, FAST/STRONG routing, BYOK, Py→JS	**design-first**	44	178	85	39
10	`judge-ready-pass`: SEC-01..12 hardening, tarot, guided UX	audit checklist	—	—	49	—
	Total		**168**	**1,011**	**716**	**173**

*"Task items" counts every checkbox in `tasks.md` (parent tasks and sub-tasks). All **716** checkboxes across the ten specs are ticked (the earlier `visual-necromancy` checkpoint markers are closed).*

How the specs were used in practice:

- **Requirements-first for new ground, design-first for additive layers.** The last two feature specs sat on top of eight prior specs, so I ran Kiro's design-first flow. The design opened with "grounding facts" read straight from the committed code (G1–G15 in `lovers-binding`, G1–G9 in `oracle-landing-polish`). Examples: that `@necro/llm` shipped only a Mock provider, and that the bundle budget was `250_000` bytes. Requirements were then derived from a design that already matched reality.
- **Traceability end to end.** Each acceptance criterion has an ID (`R17.1`). Code comments carry `Validates: Requirements R2.1, R2.2 …`, and every property test's `describe` title is tagged with the requirement it proves, for example `describe("P2 (R4.1, R4.2, R4.4): one retry per unit, FAST→STRONG only, max 2 calls", …)`.
- **Wave/DAG execution with sub-agents.** Later `tasks.md` files carry an explicit dependency graph (waves 0–8 and 0–13). Independent tasks in the same wave were sent in parallel to scoped Kiro spec-task sub-agents, building bottom-up: pure core, then components, then wiring, then e2e, then the budget gate, then docs.
- **Requirement QA.** In `oracle-landing-polish`, a requirement-detailer sub-agent and an analyze-requirements pass tightened vague criteria into checkable ones before any code existed (e.g. "the boot banner does NOT count as command output").
- **"Do no harm" as a requirement.** Each additive spec includes an explicit no-regression requirement (e.g. `lovers-binding` R32, "Do no harm to prior specs") and a final gate task, so the specs themselves blocked regressions.

Steering docs: 9 files, three inclusion modes

File	Inclusion	What it locked in
`product.md`	always	"The cards cannot lie", "revived means proven", an always-demoable Mock default
`tech.md`	always	Strict TS, ESM, pnpm monorepo, Next.js on Vercel, DOM terminal (no canvas, for a11y)
`structure.md`	always	Dependency direction `oracle ← grimoires ← coven ← web`, barrel-only imports, docs discipline
`security.md`	always	Repo code is untrusted data, prompt-injection delimiters, sandbox rules, secret masking, abuse limits
`coven-contracts.md`	fileMatch `packages/coven/**`, `packages/llm/**`	Every LLM call has a zod schema, budget, timeout, cache key and Mock fixture; the LLM may only downgrade
`grimoire-authoring.md`	fileMatch `packages/grimoires/**`	"Deterministic fixers first, LLM last"; idempotence and literal-preservation properties per fixer
`tarot-ux.md`	fileMatch UI files	The spread *is* the layout; orientation is meaning; palette; reduced-motion rules
`testing.md`	fileMatch test files	fast-check with explicit `numRuns`, requirement-tagged properties, no network/LLM in unit tests
`oracle-voice.md`	auto (by description)	Two layers always: a mystical line, then the plain technical fact

What mattered most was **scoping steering with `fileMatch`**. Kiro got LLM-contract rules only when touching the coven or LLM packages, and UX rules only when touching UI. In the build log, entry after entry reads "steering did the heavy lifting without re-prompting": the dependency direction, the palette and the error-message voice never had to be repeated in a prompt. `oracle-voice` alone wrote every error message, notice and card meaning in a consistent two-layer voice.

Agent hooks: 2 hooks that kept the loop honest

- **`auto-commit-after-task`** (`PostTaskExec`, agent action). After every spec task it builds `@necro/oracle`, runs `pnpm -r typecheck` and `pnpm -r test`, and commits only when green, with a message derived from the spec folder (`<spec-label> tasks <first>-<last>: <summary>`). It knows about test-first development: if the suite is red **only** because of planned failing tests for code not yet written, it skips the commit and continues. Any other red means stop and fix. That is why the git history reads spec by spec.
- **`block-env-writes`** (`PreToolUse`, command action). It runs `scripts/hooks/block-env-writes.mjs` before any tool call and denies writes to `.env*` files. It fails closed and enforces rule 8 of `security.md` at the agent level, so Kiro cannot accidentally write a secret file.

MCP

`.kiro/settings/mcp.json` wires two servers: **GitHub MCP** (token via `${GITHUB_PAT}` env interpolation, never committed) and **Context7**. Context7 supplied current docs during scaffolding: pnpm action-setup, Vitest 3.2 deprecations and Next.js App Router tsconfig requirements. The full 13-package monorepo skeleton came together in one session without stale-API detours.

Property-based testing: where specs paid for themselves

The 8 design documents name **173 correctness properties**, and the repo has **111 dedicated `*.pbt.test.ts` files** (fast-check, no network). Several properties caught real bugs that example tests had missed:

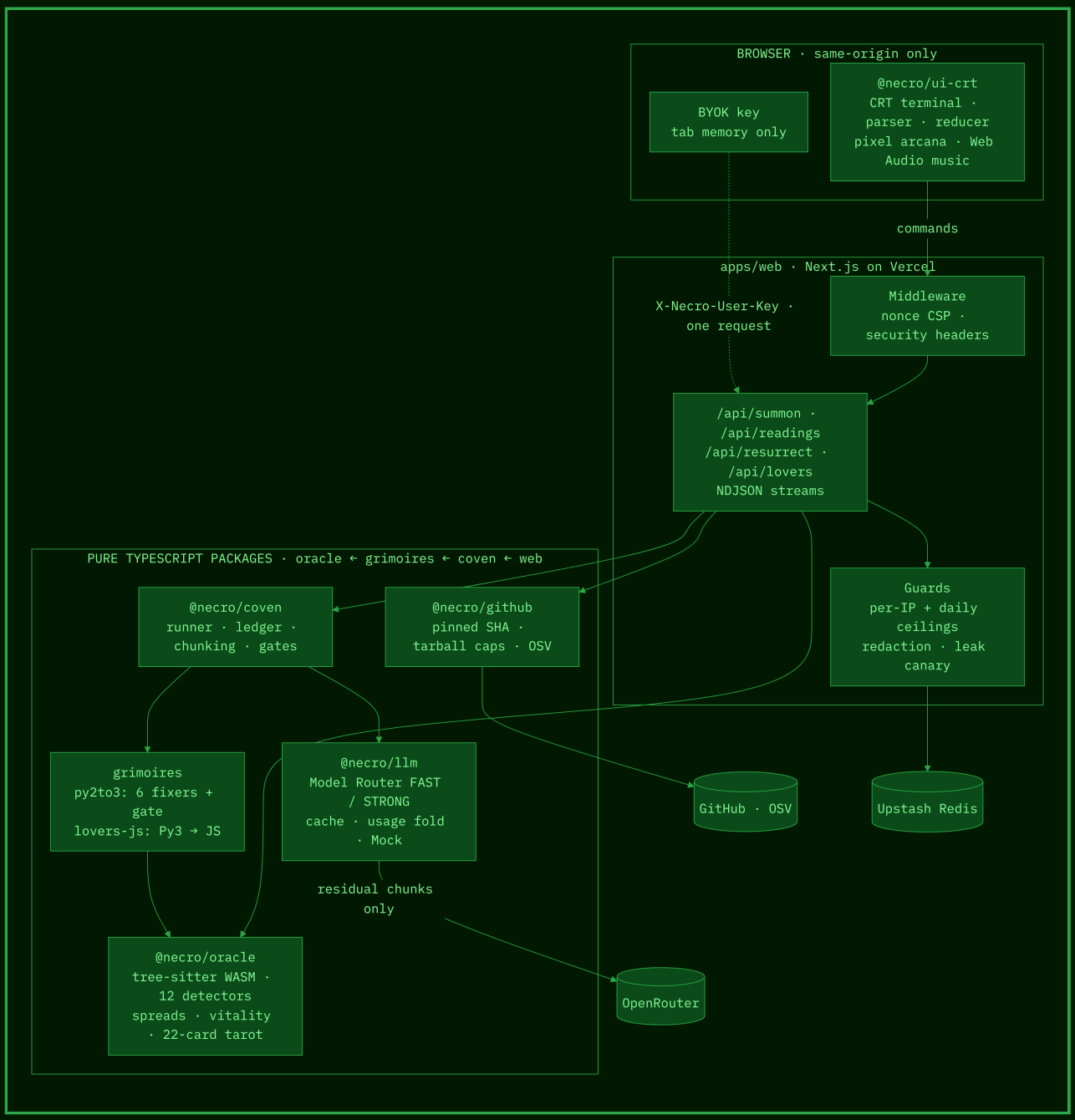
- **Lovers binding-parse property** (300 generated plans): the bridge emitted invalid JavaScript whenever a Python function or parameter was named a JS reserved word (`delete`, `new`, `class`). Fixed by making the deterministic extractor the single source of bindability truth.
- **Landing P6** (`landingVisible` is false whenever command output exists): showed that the boot banner counted as output, so the landing menu never appeared after boot.
- **Landing P10/P3** (`activate`): caught that activating a repo ran `summon` but dropped the follow-up `read fates`.
- **Usage-fold P8**: forced the cost and token accounting to be associative and commutative, with per-model sums equal to the totals.
- **Computed WCAG gate**: the tokens test calculates real sRGB contrast. It rejected the designed focus-ring color (about 1.24:1) and forced a compliant one. The test stayed the arbiter, and no threshold was weakened.

Vibe coding and specs, used deliberately

Exploratory work stayed conversational: a `tar-stream` spike with characterization tests decided between the event form and the async-iterator form before the spec committed to a design. Anything that touches correctness, security or the wire protocol went through a spec. The final **`judge-ready-pass`** was a hybrid: a 49-item audit checklist (SEC-01..12, P0–P2) run by a single agent. Every item ended either DONE with a failing-first regression test or VERIFIED with a written reason. It found real drift that vibe-built code would have shipped:

- A key typed inline as `key set sk-…` was echoed three times (now intercepted before echo or history).
- The per-reading LLM call cap was dead config (now wired).
- The LLM retry reused the rejected attempt's cache key, so retries were no-ops (now keyed separately).
- A hoisted-import gate accepted `from os import path; os.system(...)` (now fully anchored).
- WASM parse trees leaked per reading (now freed in `finally`).
- Python 2 that tree-sitter parses cleanly (`10L`, `0777`) passed validation (now AST-checked).

Architecture



ARCHITECTURE · dependency direction oracle ← grimoires ← coven ← web, enforced by tests

The core idea is **dependency direction enforced by tests**: `oracle ← grimoires ← coven ← web`. `@necro/oracle` is pure and browser-safe through a guarded `client` subpath. The only network-capable LLM module (`providers/openrouter.ts`) sits behind an injected `LlmPort`, and a static guard test proves the rest of `@necro/llm` can't reach the network. The terminal's initial JS stays under a **250,000-byte gzipped budget** (201,091 B at the last gate). Demo, sound, resurrection, background-theme and Lovers views each load as separate lazy chunks, which a post-build script verifies.

Security & BYOK

The live demo runs on my OpenRouter key, so security was treated as a feature. Twelve findings (SEC-01..12) were each closed with a failing-first test:

- **BYOK lifecycle:** `key set` opens a **masked** input. The key lives only in in-memory tab state, so reload or a second tab means no key. `key status` shows only the last 4 characters, and `key clear` drops the key and aborts in-flight requests. It is never written to localStorage, cookies, IndexedDB or logs.
- **No key echo:** typing `key set <secret>` inline is intercepted *before* echo, history or the "unknown command" path, and the masked field opens instead.
- **Transport:** the key travels in a single dedicated header (`X-Necro-User-Key`) for one request. The OpenRouter endpoint is a fixed constant (no user base URL), and the referer is set from `NECRO_PUBLIC_URL`.
- **No silent fallback:** a BYOK 401 or 402 surfaces as a typed oracle-voice error and is never quietly served from the server key. Upstream statuses are mapped distinctly (401 auth, 402 credit, 429 rate limit, 5xx outage). Only transient errors retry, and only once.
- **Logs and streams:** server logs carry an allow-listed error code plus a redacted message. Every outgoing stream line is redacted over every in-scope secret.
- **Leak canary:** a runtime sentinel is in the redaction set *and* the post-build client-bundle sweep, so a leak fails either at runtime or at build time.
- **Headers and CSP:** a per-request nonce CSP with `'strict-dynamic'` and **no** `'unsafe-inline'` (even dynamic CSS variables moved to CSSOM), plus `no-store`, `nosniff` and `no-referrer`, with no CORS reflection. `dangerouslySetInnerHTML`, `eval` and `new Function` are banned by a boundary test.
- **Spend:** per-IP hourly limits (summon 20, readings 10, resurrect 5, lovers 5), daily ceilings (LLM, Lovers and tarot each have their own counter), and a per-reading LLM call cap (default 12). A BYOK request never spends server credit.
- **Untrusted code:** repository contents are parsed only. They are wrapped in explicit delimiters as *data* in prompts, secret-masked before any prompt, and every LLM output is zod-validated and gate-checked.

AI usage done right

- **Deterministic first.** Cards are drawn by detectors, and Python 2 constructs are fixed by six idempotent, property-tested fixers. The LLM touches only the residual diagnostics, and only the chunk of the file that contains them. A property test proves each chunk's context is a strict subset of the file.
- **Dynamic model routing.** A pure `Model_Router` picks **FAST** for simple units and escalates to **STRONG** when a unit has more than 6 unresolved diagnostics, more than about 1,200 estimated tokens, or a hard construct (metaclasses, `__import__`, cross-language type mapping). Production uses `openai/gpt-4.1-mini` (FAST) and `anthropic/claude-sonnet-4.5` (STRONG), with `google/gemini-2.5-flash` and `qwen/qwen3-coder` as OpenRouter fallbacks. Model ids are pure env configuration.
- **Bounded cost per call.** Each unit gets at most 2 calls (FAST, then one escalated retry), each reading is capped (12 calls by default), every call has a token budget and timeout, and the temperature is 0. Completions are cached by a content-addressed key (purpose, schema, prompt version, resolved model, content hash) that never includes the user's key. OpenRouter's per-call `usage.cost` is folded into a per-model **"souls spent"** usage aggregate.
- **Never trust, always verify.** Outputs must parse (tree-sitter, zero error nodes), keep public signatures, and carry no Python 2 constructs. Lovers output also passes a deny-list scan. If the model fails, the deterministic result still ships.
- **Works with zero AI.** With no key configured, the app runs on a deterministic Mock provider: readings render, resurrection runs the deterministic Pre_Pass, Lovers runs from Mock fixtures, and `tarot` uses its offline reading. `pnpm smoke:llm` tests the LLM plumbing offline and makes one live call only with explicit opt-in.

Accessibility

A horror UI can still be accessible:

- **Real DOM text.** The terminal is never a canvas. A single `role="log"` live region holds the authoritative text, and pixel art and ASCII overlays are `aria-hidden` decorations. Each card has alt text that states its name, orientation and a meaning of up to 120 characters.
- **axe with zero violations** in Playwright e2e, keyboard-only operation (the landing menu responds to arrow keys, Enter/A, B/D and H and never steals modified keys), and layouts that work at **320 px wide and 200 % zoom**.
- **Motion and sound safety.** `prefers-reduced-motion` and `motion off` render everything instantly, and a Flash_Limit property keeps motion zones to 2 or fewer luminance changes per second. Sound is off by default, gesture-gated, and the master gain is capped.
- **No-JS note** for browsers without JavaScript.

Challenges

- **WebAssembly on serverless.** Tree-sitter grammars are `.wasm` files that Next.js output tracing and Vercel's packaging both fought us on. Spec `fix-wasm-tracing` plus a post-build `check:wasm-trace` gate landed first; production then hit a second failure when traced paths went through pnpm symlinks (`packages/oracle/node_modules/tree-sitter-*`), which Vercel rejects. Hotfixes `cafb341` and `619ebc4` copy the grammars as **real files** into `packages/oracle/dist/grammars/`, make the gate symlink-aware, and pin the Vercel runtime to **Node 22.x**. Loading the JS grammar lazily still keeps Python-only routes from depending on it.
- **A strict CSP versus a pixel-art UI.** Removing `'unsafe-inline'` meant moving every dynamic style (card scale, glitch offsets, ritual timing) to CSSOM custom properties through a `useCssVars` hook.
- **Encoding hazards.** BOM-prefixed `package.json` files broke PostCSS, and mojibake em-dashes broke SWC even though vitest and tsc accepted them. Both were found by the final full gate matrix rather than by unit tests.
- **Honest e2e.** Four Playwright failures I first dismissed as "pre-existing" turned out to be real bugs: an illegal card id used as the Lovers precondition, and a word-break space lost in wrapped log text. Both were fixed at the root.
- **Layout in a real browser.** Follow-up polish (`be1c9ae`, then `5930ae6`) put results inside the 100dvh scroll pane so they no longer sit as a solid block over the card art, capped diff context, kept captions from overlapping, and added a Playwright test (`layout-scroll.e2e.ts`) that replays the resurrect path. Auto-scroll to the very end is still imperfect on some screens (see Known issues).

Accomplishments we're proud of

- **3,091 unit and property tests passing across 301 test files, plus 52 Playwright e2e tests**, with typecheck, lint at 0 warnings, the production build, the bundle budget and the symlink-aware WASM-trace check all green on commit `5930ae6` (`fix(ux-polish)`: 100dvh layout with results inside the scroll pane, `explain past|present|future`, demo listing encoding, layout-scroll e2e — after `be1c9ae` sticky-bottom scroll and `619ebc4` grammar WASM as real files for Vercel).
- 168 requirements and 1,011 acceptance criteria, traceable to code and to 173 named properties; 716/716 spec task checkboxes ticked.

- **Production proof (2026-11-09, https://necro-oracle.vercel.app):** a real `tomaes/pytarl` reading streams SUMMON→DOWNLOAD→EXTERNAL→READING; a live resurrect of `pytarl.py` (~30 KB Python 2 roguelike) fully converts with zero residuals and **no LLM cost**, vitality **0 → 12**, Judgement removed, ~3 s (the deterministic Pre_Pass did the work); a later live Lovers binding (`lovers pytarl.py javascript`) bound **7 functions** and returned `pythonHeart`, `jsBeloved`, `contractSchema`, `pythonTests` and `jsTests` for **6,418 tokens** and **0.0842 USD**; an earlier Lovers run on the same file routed to STRONG `anthropic/claude-sonnet-4.5` (429 prompt + 1,200 completion tokens, **0.019 USD**); demo mode covers resurrect and Lovers with the Mock LLM.
- A finished-feeling product: boot sequence, Game Boy menu, pixel-art arcana, Necronomicon ritual, original synthesized soundtrack and offline demo autoplay, all inside a 250 KB first-load budget.
- An AI pipeline whose LLM can only make things *better*. It can never draw a card, skip a gate or leak a key. FAST/STRONG routing and residual-only chunking keep spend tiny; BYOK keys stay in tab memory only.

What we learned

- **Specs pay off when they are written against the real code.** Design-first with grounding facts beat requirements-first for every additive layer.
- **Steering is most effective when scoped narrowly.** A few precise `fileMatch` rules outperformed a single large always-on prompt.
- **Hooks turn discipline into a default.** "Commit only when green, but allow planned test-first red" kept the build honest without slowing it down.
- **Property tests are the best reviewer** for anything generated, whether code from an LLM or code from an agent.

Lessons learned: working efficiently with Kiro

Spec-driven development — requirements, then design, then tasks — gave this project its structure and more than 3,000 tests. The expensive lesson was not the method. It was prompt scope.

Early on, broad prompts let Kiro spawn many sub-agents and rerun the full suite after every small change. A single feature pass could burn hundreds of credits and leave scattered, hard-to-review results.

I switched to short, self-contained prompts: **NO subagents**, a minimal diff, run only the touched tests plus lint, typecheck and build, no full suite, no e2e, and no commit. I ran the full gates myself afterwards. With that shape, a focused fix cost about **8–33 credits** — the Vercel symlink grammar fix was about 27, a duplicate-render fix about 8.

jsdom unit tests cannot catch real layout and scroll bugs. One polish passed every unit test and still failed in production: after resurrect or Lovers, the result view did not scroll to the end. The remedy is a real-browser Playwright test that replays the exact user flow, written to fail first (`layout-scroll.e2e.ts` landed in `5930ae6`).

When an agent iterates blind on the same failing assertion several times, stop and re-scope. One concern per prompt, and after two failed attempts the agent should stop and report rather than keep guessing.

Known issues

On some screens, the result view after `resurrect` or `lovers` does not yet auto-scroll all the way to the end. The unified diff and the **DOWNLOAD PATCH** / **DOWNLOAD PAIR** actions are present just below — scroll the output pane to reach them.

What's next

- **Wire the model-backed tarot reading** (`/api/tarot`) into the terminal. The one-call FAST service (≤400 output tokens, its own daily ceiling, offline fallback) is already built and tested in `@necro/coven`; the ceremonial draw ships today with the deterministic offline text. Listed on the repo Roadmap until the route is live.
- **Sandboxed differential verification** (repo Roadmap, L1 Flesh). Run the original and resurrected code side by side inside a per-ritual micro-VM. An optional E2B `py_compile` provider already exists behind the `SandboxPort`.
- **Pull requests to forks** (repo Roadmap, L1 Flesh). Open a real PR against a fork of the target repo so "REVIVED" means proven and delivered.
- **Chaos mode.** The oracle deliberately rots a healthy repo (seeded, reversible mutations) and challenges you or an agent to resurrect it under the gates. It would work as a game and as a benchmark for migration agents.
- **The oracle everywhere.** `necro-mcp` (so Kiro itself can consult the oracle), a `necro` CLI and a GitHub Action that reads tarot on every PR. The packages are scaffolded, and the core is already a reusable library.
- **More grimoires:** packaging (`setup.py` to `pyproject`), and jQuery to vanilla ES2022.

Categories and prizes we're targeting

- **Category: Frankenstein.** The Necro-Oracle stitches together pieces that have no business working together: a 1986 green-phosphor terminal in Game Boy pixel art, 15th-century tarot, tree-sitter WebAssembly static analysis, a deterministic codemod engine, LLM routing across four model families, and a Python-to-JavaScript binding that literally stitches two runtimes together through a JSON contract. The result is useful rather than a gimmick: the tarot spread is the information architecture.
- It also fits the **Resurrection** brief: it brings dead technology (Python 2 code, 8-bit handheld aesthetics, CRT terminals) back to life with today's tooling. And it fits the **Costume Contest** brief: the spooky UI *enhances function*, because orientation, position and the vitality gauge carry the diagnosis.
- **Overall prizes:** strong on all five criteria. *Impact:* legacy-code triage and migration. *Implementation:* deep, varied Kiro usage. *Design:* a complete, accessible product. *Innovation:* tarot as the information architecture for code health. *Presentation:* a memorable demo.
- **Bonus prizes:** Best Demo (the 3-minute video), Blog Post and Social Blitz (`#KiroweenHackathon26`). Eligibility for the remaining bonuses depends on the submitter's details.

Built with

TypeScript (strict) · Node 22 · pnpm workspaces · Next.js (App Router) · React · Vercel · web-tree-sitter (Python and JavaScript grammars, WASM) · zod · fast-check · Vitest · Playwright · axe · Upstash Redis · OpenRouter · Web Audio API · Kiro (specs, steering, hooks, MCP, sub-agents)

Try it

- **Live:** https://necro-oracle.vercel.app (no sign-up; Node 22.x on Vercel). Useful URL flags: `?boot=skip` lands on the menu immediately; `?ritual=force` always plays the full Necronomicon ritual.
- **Suggested live path:** activate `tomaes/pytarl` → `explain 2` (or `explain present`) → `resurrect` → `lovers pytarl.py js`.

- **Offline-proof path:** `demo lich-py2` replays a recorded reading with no network, then autoplays a tarot draw; demo resurrect and Lovers run on the Mock LLM.
- **Run locally:** `pnpm install && pnpm -r build && pnpm dev` (Node 22, pnpm 10). No keys are needed; it defaults to the deterministic Mock provider.

Tarot card art derives from the Rider–Waite–Smith deck illustrated by Pamela Colman Smith (1909), public domain. All music is synthesized at runtime; there are no audio files.